

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Обеспечение информационной безопасности мобильных устройств»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Создание приложения и поиск URL, конечных точек и секретов в APK-файлах (Flutter)»

Выполнил:

Бардышев Артём Антонович,
студент группы N3346



(подпись)

Проверил:

Федоров Иван Романович,
преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение.....	3
1 Ход работы.....	4
1.1 Особенности работы приложения.....	6
1.2 Получение APK-файла двумя способами: с обфускацией и без обфускации.....	6
1.3 Сканирование приложения утилитой apkleaks и проверка через VirusTotal	7
Заключение.....	9
Список использованных источников	10

ВВЕДЕНИЕ

Цель работы – разработка мобильного приложения на Flutter и изучение методов обеспечения его безопасности, а также получение практических навыков анализа защищённости Android-приложений.

Для достижения поставленной цели необходимо решить следующие задачи:

- ☐ Разработать мобильное приложение на Flutter, аналогичное приложению из лабораторной работы №3;
- ☐ Реализовать в приложении три экрана: экран с выводом ФИО студента, экран с данными из REST API и экран с формой обратной связи с использованием локальной базы данных;
- ☐ Реализовать запрет на создание скриншотов экрана в приложении;
- ☐ Получить APK-файл двумя способами: с обфускацией и без обфускации;
- ☐ Декомпилировать полученные APK-файлы с помощью ArkTool и выполнить их сравнение;
- ☐ Просканировать разработанное приложение с использованием утилиты arkleaks;
- ☐ Проверить разработанное приложение с помощью сервиса VirusTotal.

1 ХОД РАБОТЫ

Для начала в Android Studio был разработан проект мобильного приложения на фреймворке Flutter. Приложение является аналогом ранее созданного проекта на языке Kotlin (лабораторная работа №3), однако реализовано с использованием кроссплатформенного подхода.

После создания проекта была настроена структура приложения и реализована навигация между экранами с помощью нижней панели (BottomNavigationBar). В результате получилось, что приложение включает несколько экранов с различным функционалом.

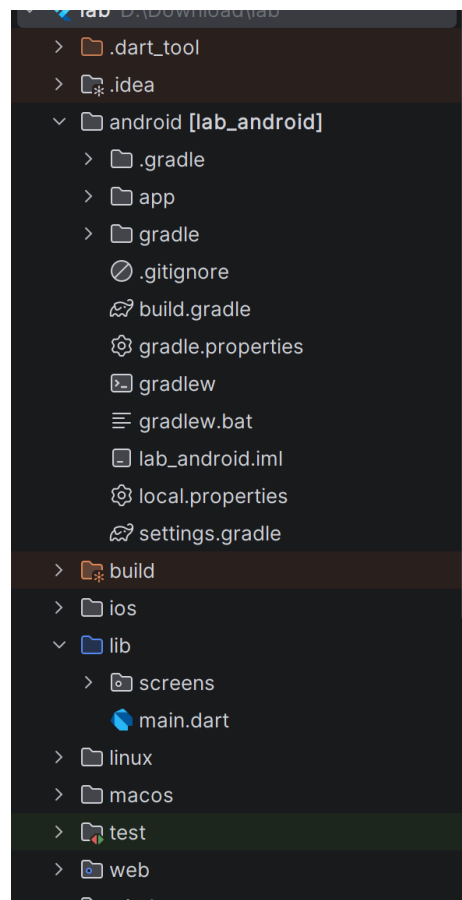


Рисунок 1 - Структура приложения Flutter

- main.dart – основной файл приложения, содержащий точку входа и инициализацию компонентов;
- экран «О студенте», отображающий информации о студенте;
- экран «Данные из REST API», отвечающий за получение и вывод данных с удалённого сервера;
- экран «Обратная связь», содержащий форму для ввода и сохранения пользовательских данных;

- локальная база данных SQLite – используется для хранения отправленных сообщений;
- навигационная панель BottomNavigationBar – обеспечивает переключение между экранами.

```
main.dart x pubspec.yaml feedback_screen.dart
1 import 'package:flutter/material.dart';
2 import 'screens/name_screen.dart';
3 import 'screens/api_screen.dart';
4 import 'screens/feedback_screen.dart';
5
6 > void main() async {...}
7
8 class MyApp extends StatelessWidget {
9   const MyApp({super.key});
10
11   @override
12   Widget build(BuildContext context) {
13     return MaterialApp(
14       title: 'MP Flutter',
15       debugShowCheckedModeBanner: false,
16       theme: ThemeData(
17         colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
18         useMaterial3: true,
19       ),
20       home: const MainScreen(),
21     );
22   }
23 }
24
25 class MainScreen extends StatefulWidget {
26   const MainScreen({super.key});
27
28   @override
29   State<MainScreen> createState() => _MainScreenState();
30 }
31
32 class _MainScreenState extends State<MainScreen> {
33   int _currentIndex = 0;
34
35   final List<Widget> _screens = const [
36     NameScreen(),
37   ];
```

Рисунок 2 - Код main.dart, 1 часть

```
main.dart x pubspec.yaml feedback_screen.dart
38
39 final List<Widget> _screens = const [
40   NameScreen(),
41   ApiScreen(),
42   FeedbackScreen(),
43 ];
44
45 @override
46 Widget build(BuildContext context) {
47   return Scaffold(
48     body: IndexedStack(
49       index: _currentIndex,
50       children: _screens,
51     ),
52     bottomNavigationBar: BottomNavigationBar(
53       currentIndex: _currentIndex,
54       selectedItemColor: Colors.indigo,
55       onTap: (index) => setState(() => _currentIndex = index),
56       items: const [
57         BottomNavigationBarItem(
58           icon: Icon(Icons.person),
59           label: '0 студенте',
60         ),
61         BottomNavigationBarItem(
62           icon: Icon(Icons.cloud_download),
63           label: 'API данные',
64         ),
65         BottomNavigationBarItem(
66           icon: Icon(Icons.feedback),
67           label: 'Обратная связь',
68         ),
69       ],
70     ),
71   );
72 }
73 }
```

Рисунок 3 - Код main.dart, 2 часть

1.1 Особенности работы приложения

После реализации функционала приложение было запущено на эмуляторе. Появились те самые три окна. Переключение между окнами осуществляется с помощью нижнего меню навигации.

Интерфейс реализован с использованием виджетов Column, Card, Icon и Text. Информация представлена в виде карточки с аватаром.

Экран выполняет HTTP-запрос к публичному API с использованием пакета http. Реализована обработка состояний загрузки, ошибок, а также кнопка обновления данных.

Экран содержит форму с полями Имя, Email и Сообщение. Данные сохраняются в локальной базе SQLite с использованием пакета sqflite. Реализован просмотр истории сообщений через переключатель в AppBar.

Дополнительно в приложении реализован механизм защиты от создания скриншотов. Для этого используется пакет flutter_windowmanager с флагом FLAG_SECURE, который устанавливается при запуске приложения.

1.2 Получение APK-файла двумя способами: с обфускацией и без обфускации

Далее необходимо получить APK-файл разработанного приложения двумя способами: с обфускацией и без неё. Обфускация используется для усложнения анализа и защиты кода приложения.

Для сборки APK без обфускации использовались следующие команды: «flutter build apk –debug» или «flutter build apk --release --no-shrink».

После выполнения команды формируется APK-файл. Переименуем его.

Для сборки APK с обфускацией используется следующая команда: «flutter build apk --obfuscate --split-debug-info=./debug_info».

Также был настроен файл proguard-rules.pro, содержащий правила для корректной работы библиотек Flutter, flutter_windowmanager и sqflite после обфускации. Переименуем его.

После получения APK-файлов была выполнена их декомпиляция с использованием утилиты ApkTool, предназначенной для анализа Android-приложений.

Декомпиляция выполнялась следующими командами: «java -jar apktool.jar d app-release-no-obf.apk -o decompiled_no_obf» и «java -jar apktool.jar d app-release-obf.apk -o decompiled_obf».

Сравним пространство имен и заметим, что с обфускацией пространство имен выглядит по-другому, функции и классы имеют измененные названия, для снижения читаемости кода и повышения безопасности.

Также код с обфускацией сложнее читать и понимать логику приложения, так как названия функций имеют другие названия и строки имеют случайные символы.

В результате сравнения было установлено, что:

- ☐ в версии без обфускации имена классов и методов остаются читаемыми и отражают структуру исходного кода;
- ☐ в версии с обфускацией имена заменяются на короткие и неинформативные
- ☐ логика приложения становится значительно сложнее для анализа.

Таким образом, обфускация существенно повышает уровень защиты приложения от реверс-инжиниринга.

1.3 Сканирование приложения утилитой arkleaks и проверка через VirusTotal

Следующим этапом стало сканирование APK-файла с помощью утилиты arkleaks, предназначенной для поиска потенциально чувствительных данных.

Установка выполняется через команду: «pip install arkleaks». Запуск сканирования: «arkleaks -f app-release.apk».

В результате анализа были обнаружены:

- ☐ URL публичного API;
- ☐ внутренние пути Flutter/Dart.

При этом конфиденциальные данные (ключи, токены) обнаружены не были, что свидетельствует о корректной реализации безопасности.

На заключительном этапе APK-файл был проверен с помощью сервиса VirusTotal, который анализирует файлы с использованием множества антивирусных движков.

После загрузки APK-файла на сайт <https://www.virustotal.com> было установлено, что приложение не содержит вредоносного кода. Все проверки завершились с положительным результатом.

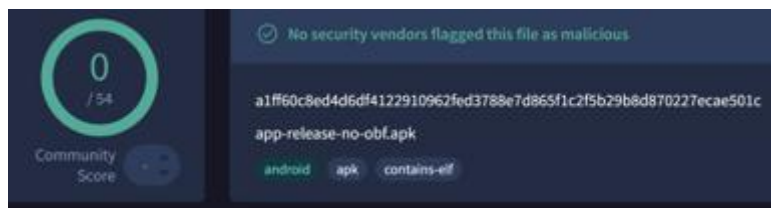


Рисунок 4 – Результат проверки VirusTotal

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработано мобильное приложение на Flutter, включающее три основных экрана: экран с информацией о студенте, экран с данными, полученными через REST API, и экран с формой обратной связи с сохранением данных в локальной базе SQLite. Дополнительно реализована защита приложения от создания скриншотов с использованием механизма FLAG_SECURE.

Получены APK-файлы двумя способами: с обфускацией и без неё. С использованием утилиты ArkTool выполнено декомпилирование файлов и проведено их сравнение. В результате установлено, что обфускация существенно усложняет анализ кода, делая имена классов и методов нечитаемыми.

Также разработанное приложение просканировано с помощью утилиты apkleaks. В ходе анализа были обнаружены только общедоступные данные, такие как URL API, при этом конфиденциальная информация выявлена не была.

На заключительном этапе APK-файл проверен с помощью сервиса VirusTotal. Результаты анализа показали отсутствие вредоносного кода в приложении.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ